

Scallion Pancake Party

Bohan đang tổ chức một bữa tiệc. Vì anh ấy rất thích bánh hành lá, nên anh ấy quyết định mua chúng từ cửa hàng gần đó cho bữa tiệc của mình. Cửa hàng bán N loại bánh hành lá với các hương vị đôi một khác nhau, được đánh số từ 0 đến $N - 1$. Bohan quyết định mỗi loại sẽ mua N bánh hành lá, tổng cộng là N^2 bánh hành lá. Mỗi bánh hành lá được chia thành K lát giống hệt nhau và cho vào một túi. Các túi được đánh số từ 0 đến $N^2 - 1$. Gọi $F[i]$ là hương vị bánh hành lá của túi i .

Trong bữa tiệc, Bohan mời tất cả các khách cùng chơi một trò chơi. Trò chơi diễn ra như sau:

Đầu tiên, Bohan chọn N khách và đưa họ vào các phòng được đánh số $0, 1, \dots, N - 1$ sao cho mỗi phòng chỉ có đúng một khách. Những khách còn lại sẽ ở ngoài cho đến khi trò chơi kết thúc. Tất cả N phòng đều trông giống hệt nhau, vì vậy khách không thể biết mình đang ở phòng nào.

Tiếp theo, với mỗi phòng i ($0 \leq i < N$), Bohan vào phòng i và bí mật nói cho khách biết một số nguyên $P[i]$, biểu thị cho một **hương vị bị cấm**. Anh ta có thể nói hoặc không nói cho khách biết họ đang ở phòng nào. Bohan đảm bảo rằng $[P[0], P[1], \dots, P[N - 1]]$ là một hoán vị của $[0, 1, \dots, N - 1]$.

Sau đó, có N vòng. Trong vòng thứ i , Bohan lần lượt mang các túi $0, 1, \dots, N^2 - 1$ theo thứ tự vào phòng $i - 1$. Khi khách ở phòng $i - 1$ nhận được túi j , họ sẽ biết:

- $F[j]$, hương vị bánh hành lá có trong túi j , và
- số lát bánh còn lại trong túi.

Trước khi nhận túi tiếp theo, khách ở phòng $i - 1$ phải quyết định ăn bao nhiêu lát từ túi hiện tại. Số lát họ có thể ăn phụ thuộc vào tình huống:

- Nếu $P[i - 1] \neq F[j]$, khách có thể lấy một số lượng lát bất kỳ từ túi và ăn.
- Nếu $P[i - 1] = F[j]$, khách không được phép ăn bất kỳ lát nào.

Lưu ý rằng khách không biết trước đây $F[0], F[1], \dots, F[N^2 - 1]$.

Sau tất cả N vòng, khách bên ngoài sẽ được xem dãy các túi. Khi nhìn thấy các túi, họ sẽ biết được hương vị bánh hành lá và số lượng lát còn lại trong mỗi túi. Với thông tin này, họ phải xác định giá trị của $P[0], P[1], \dots, P[N - 1]$.

Khách mời được phép trao đổi trước khi trò chơi bắt đầu. Họ cũng biết trước giá trị của N và K . Mục tiêu của bạn là thực hiện một chiến lược sao cho khách mời bên ngoài luôn có thể xác định chính xác giá trị của $P[0], P[1], \dots, P[N - 1]$.

Chi tiết cài đặt

Bạn cần xây dựng ba hàm.

Bạn cần xây dựng hai hàm sau đây cho khách ở các phòng $0, 1, \dots, N - 1$.

```
void init(int N, int K, int p, int r)
```

Giả sử khách đang ở phòng i .

- N : số lượng hương vị bánh hành lá.
- K : số lát bánh hành lá của mỗi loại trước khi trò chơi bắt đầu.
- $p = P[i]$: hương vị bị cấm ăn trong phòng i .
- Nếu Bohan quyết định nói cho khách biết họ đang ở phòng nào, thì $r = i$. Nếu không, $r = -1$.

```
int strategy(int b, int f, int s)
```

Giả sử khách đang ở phòng i .

- b : số hiệu túi hiện tại.
- $f = F[b]$: hương vị bánh hành lá của túi b .
- s : số lát bánh còn lại trong túi b .
- Hàm này cần trả về một số nguyên không âm x , biểu thị số lát bánh mà khách quyết định ăn.
 - Nếu $f \neq P[i]$, thì $0 \leq x \leq s$.
 - Nếu $f = P[i]$, thì $x = 0$.

Bạn cần xây dựng hàm sau cho khách bên ngoài.

```
std::vector<int> guess(int N, int K, std::vector<int> F,  
                      std::vector<int> S)
```

- N : số lượng hương vị bánh hành lá.
- K : số lát của mỗi loại bánh hành lá trước khi trò chơi bắt đầu.
- F : một mảng có kích thước N^2 , trong đó $F[i]$ là hương vị của bánh hành lá trong túi i .
- S : một mảng có kích thước N^2 , trong đó $S[i]$ là số lát còn lại trong túi i sau tất cả N vòng chơi.

- Hàm này sẽ trả về một vector P có độ dài N , trong đó $P[i]$ là số được giao cho khách trong phòng i .

Mỗi trường hợp thử nghiệm bao gồm T trò chơi độc lập.

Trong quá trình đánh giá, đối với mỗi trò chơi trong T trò chơi, sẽ có $N + 1$ tiến trình. Với mỗi i sao cho $0 \leq i < N$, tiến trình i đại diện cho khách trong phòng i . Tiến trình N đại diện cho khách bên ngoài. Với mỗi i sao cho $0 \leq i < N$, tiến trình $(i + 1)$ bắt đầu sau khi tiến trình i kết thúc.

Đối với các tiến trình từ 0 đến $N - 1$:

- Hàm `init` được gọi đúng một lần.
- Hàm `strategy` được gọi N^2 lần sau hàm `init`.
 - Đảm bảo rằng đối với lần gọi thứ j , $b = j - 1$.

Đối với tiến trình N :

- Hàm `guess` được gọi đúng một lần.

Trình chấm có thể xen kẽ các tiến trình từ các trò chơi khác nhau, nhưng đảm bảo rằng thứ tự tương đối của các tiến trình trong mỗi trò chơi riêng lẻ được bảo toàn.

Các ràng buộc

- $1 \leq T \leq 400$.
- $2 \leq N \leq 30$.
- Tổng của N^3 trên tất cả các trò chơi trong một trường hợp thử nghiệm không vượt quá 27 000.
- $1 \leq K \leq N$
- $K \in \{1, 3, N\}$.
- $[P[0], P[1], \dots, P[N - 1]]$ là một hoán vị của $[0, 1, \dots, N - 1]$.
- $0 \leq F[j] < N$ với mỗi j thỏa mãn $0 \leq j < N^2$.
- i xuất hiện đúng N lần trong $[F[0], F[1], \dots, F[N^2 - 1]]$ với mỗi i thỏa mãn $0 \leq i < N$.
- Trình chấm **không thích ứng**, nghĩa là, các giá trị $[P[0], P[1], \dots, P[N - 1]]$ và $[F[0], F[1], \dots, F[N^2 - 1]]$ được cố định trước khi hàm `init` được gọi lần đầu tiên.

Các subtask

Subtask	Điểm	Các ràng buộc thêm
1	8	$K = 1, N = 2$.
2	7	$K = 1$ và Bohan quyết định nói cho mọi người biết họ đang ở phòng nào.
3	14	$K = 1$ và $[F[i \cdot N], F[i \cdot N + 1], \dots, F[i \cdot N + N - 1]]$ là một hoán vị của $[0, 1, \dots, N - 1]$ với mỗi i thỏa mãn $0 \leq i < N$.
4	18	$K = N$.
5	21	$K = 3, N \geq 3$.
6	32	$K = 1$.

Ví dụ

Xét tình huống trong đó $T = 1$ và $N = 2, K = 2, P = [1, 0], F = [1, 0, 0, 1]$ cho một trò chơi trong trường hợp thử nghiệm.

Gọi $S[i]$ là số lát bánh còn lại trong túi i . Ban đầu, $S = [2, 2, 2, 2]$.

Giả sử các khách xác định chiến lược dưới đây trước khi trò chơi bắt đầu:

- Đối với khách trong phòng, nếu họ được phép ăn chiếc bánh hành lá hiện tại:
 - Nếu chiếc bánh hành lá hiện tại là chiếc đầu tiên họ được ăn, thì họ sẽ ăn hết tất cả các lát còn lại.
 - Nếu không, họ chỉ ăn một lát.
- Đối với khách bên ngoài:
 - Nếu $S = [0, 0, 1, 1]$, thì họ sẽ đoán rằng $P = [1, 0]$.
 - Nếu không, họ sẽ đoán rằng $P = [0, 1]$.

Trò chơi diễn ra như sau:

Đầu tiên, trình chấm khởi động tiến trình 0 đại diện cho khách trong phòng 0 và gọi `init(2, 2, 1, -1)`. Sau khi khởi tạo, trình chấm thực hiện các lệnh gọi sau:

Gọi hàm	Giá trị trả về
<code>strategy(0, 1, 2)</code>	0
<code>strategy(1, 0, 2)</code>	2
<code>strategy(2, 0, 2)</code>	1
<code>strategy(3, 1, 2)</code>	0

Lần gọi thứ nhất và thứ tư phải trả về 0 vì $P[0] = F[0] = F[3] = 1$.

Sau khi quá trình này kết thúc (vòng 1 kết thúc), $S = [2, 0, 1, 2]$.

Tiếp theo, trình chấm bắt đầu quá trình 1 đại diện cho khách 1 và gọi `init(2, 2, 0, -1)`. Sau khi khởi tạo, trình chấm thực hiện các lệnh gọi sau:

Gọi hàm	Giá trị trả về
<code>strategy(0, 1, 2)</code>	2
<code>strategy(1, 0, 0)</code>	0
<code>strategy(2, 0, 1)</code>	0
<code>strategy(3, 1, 2)</code>	1

Lưu ý rằng cuộc gọi thứ hai và thứ ba phải trả về 0 vì $P[1] = F[1] = F[2] = 0$.

Sau khi quá trình này kết thúc (vòng 2 kết thúc), $S = [0, 0, 1, 1]$.

Cuối cùng, trình chấm bắt đầu quá trình 2 đại diện cho các khách bên ngoài. Trình chấm thực hiện cuộc gọi sau:

Gọi hàm	Giá trị trả về
<code>guess(2, 2, [1, 0, 0, 1], [0, 0, 1, 1])</code>	<code>[1, 0]</code>

Các vị khách bên ngoài đã đoán đúng hoán vị. Do đó, trường hợp thử nghiệm này được đánh giá là đúng.

Lưu ý rằng phương pháp này không phải lúc nào cũng giúp các vị khách bên ngoài đoán đúng hoán vị.

Gợi đính kèm cho bài toán này chứa một mẫu đầu vào khác.

Trình chấm mẫu

Trình chấm mẫu chỉ hỗ trợ **một trò chơi cho mỗi trường hợp thử nghiệm**.

Định dạng đầu vào:

```
N K
P[0] P[1] ... P[N-1]
F[0] F[1] ... F[N*N-1]
reveal
```

- `reveal` bằng 0 hoặc 1.
 - Nếu `reveal` bằng 0, trình chấm sẽ coi như Bohan không nói số phòng của ai.
 - Nếu `reveal` bằng 1, trình chấm sẽ coi như Bohan nói số phòng của mọi người.

Nếu vector được trả về bởi `guess` khớp với $[P[0], P[1], \dots, P[N - 1]]$, trình chấm mẫu sẽ in ra

```
Accepted
```

Ngoài ra, trình chấm sẽ in ra các lệnh gọi hàm đã thực hiện để phục vụ mục đích gỡ lỗi.